Lyapunov Exponents as Predictability Diagnostics for Differentiable Simulators and Learned World Models

Benjamin Strittmatter

June 2026

Abstract

Draft in progress.

Contents

1 Introduction

[To be written last.]

2 Background

2.1 Lyapunov exponents and λ_1

Chaos is a concept borrowed from information theory, which quantifies how inherently difficult it is to make predictions about the future of dynamical systems. Mathematically speaking, a hallmark of chaos is the exponential divergence of nearby trajectories:

$$x(t) - \tilde{x}(t) \sim e^{\lambda t} \,, \tag{1}$$

where $\tilde{x}(t) = x(t) + \delta(t)$ is related to the original trajectory by a small pertubation $\delta(t)$. The exponential rate of growth is called the "Lyapunov exponent". Goal of this work is to demonstrate that the Lyapunov exponent plays an important role in characterising the predictability horizons in robotic learning.

To obtain a language that is more closely related to the robotics framework we want to study, it is advantageous to work in the tangent space picture. In practice, the trajectory $x(t)$ is determined by an ODE $\dot{x} = f(x)$, for some function $f(x(t))$. We can linearise the ODE for a perturbation $x + \delta$, and arrive at the expression

$$\dot{\delta} = Df(x)\,\delta \,, \tag{2}$$

with $Df(x)$ the Jacobian matrix. Mirroring the mechanics of a differentiable simulator, we discretise the flow with a fixed step dt. This gives a one-step map $x_{t+1} = F(x_t)$, whose linearisation propagates the perturbation by the one-step Jacobian J_t:

$$\delta_{t+1} = J_t \delta_t, \qquad J_t = \frac{\partial x_{t+1}}{\partial x_t}. \tag{3}$$

Solving the discrete recursion gives $\delta_t = M_t\,\delta_0$, propagated by the *rollout Jacobian*

$$M_t = \frac{\partial x_t}{\partial x_0} = \prod_{s<t} J_s, \tag{4}$$

the fundamental matrix of the linearised flow. The *largest Lyapunov exponent* is the asymptotic growth rate of a generic infinitesimal perturbation,

$$\boxed{\lambda_1 = \lim_{t\to\infty} \frac{1}{t}\, \ln \frac{\|M_t\,\delta_0\|}{\|\delta_0\|}}, \tag{5}$$

a limit that exists for almost every initial condition by Oseledets' multiplicative ergodic theorem [1]. The order of limits matters—δ_0 infinitesimal first, then $t \to \infty$—which is why the exponent is a property of the linearised flow rather than of two finite trajectories, whose separation saturates once it is no longer small. A positive λ_1 is the operational signature of chaos: an initial uncertainty ε grows to order unity within the *predictability* (or *Lyapunov*) *time* $\tau \sim \lambda_1^{-1} \ln(1/\varepsilon)$, so $1/\lambda_1$ sets the horizon beyond which the trajectory is effectively unpredictable. A central claim of this work is that the *same* λ_1 governs the predictability horizon of both computational objects of modern robot learning—differentiable simulators (Part A) and learned world models (Part B).

The perturbation directions that grow fastest are the leading singular vectors of M_t; its singular values $\sigma_i(M_t) \sim e^{\lambda_i t}$ define the full *Lyapunov spectrum* $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$. (Singular rather than eigenvalues: M_t is non-normal, and a pure shear has unit eigenvalues yet still stretches space.) The sum of the spectrum measures the mean exponential growth of phase-space volume [2],

$$\sum_{i=1}^{n} \lambda_i = \langle \nabla \cdot f \rangle = \frac{1}{dt} \langle \ln |\det J| \rangle, \tag{6}$$

the time-averaged divergence of the flow. The systems we study are Hamiltonian, so Liouville's theorem gives $\nabla \cdot f = 0$ and the spectrum sums to zero: phase-space volume is conserved. This volume-preservation constraint is a structural fingerprint that a faithful model must reproduce—a theme we return to in Part B (Section 5)—and serves throughout as a consistency check on the estimator.

Reading the *full spectrum* off an explicitly formed M_t is numerically hopeless: the dynamic range $\sigma_1/\sigma_n \sim e^{(\lambda_1 - \lambda_n)t}$ over- or under-flows and every column collapses onto the dominant direction, burying the sub-dominant exponents within a few thousand steps. We instead use the standard Benettin/QR estimator [3, 4]: evolve an orthonormal frame Q under the one-step Jacobians, re-orthonormalise at each step through a factorisation $J_t Q = Q' R$, and accumulate the logarithms of the diagonal entries of R; the running averages converge to the exponents λ_i (reported per unit time, after dividing by dt), with convergence guaranteed by Oseledets. A single such estimator is used throughout, calibrated against known answers: it returns zero to machine precision on a harmonic oscillator and recovers $\lambda_1 \approx 0.906$ for the Lorenz system, as detailed in Section 3.2.

2.2 Jacobians and differentiable simulation

A *differentiable simulator* is a physics engine whose one-step map $x_{t+1} = F(x_t)$ is implemented so that derivatives propagate through it: applying reverse-mode automatic differentiation to a rollout returns the gradient of a trajectory objective with respect to the initial state, the physical parameters, or a sequence of control inputs. We use NVIDIA Warp [5] for every simulator in this work; the implementation details are deferred to Section 3.1.

The object such a simulator differentiates is exactly the rollout Jacobian of Section 2.1. For a scalar loss $L(x_T)$ evaluated at the end of a T-step rollout, the chain rule gives

$$\frac{\partial L}{\partial x_0} = \frac{\partial L}{\partial x_T} M_T, \qquad M_T = \prod_{t<T} J_t, \tag{7}$$

with M_T the rollout Jacobian of Eq. (4). Reverse-mode differentiation evaluates this product right-to-left, multiplying in one J_t per step as it walks back along the stored trajectory. Every gradient a differentiable simulator produces is therefore a product of one-step Jacobians—the same product that governs the growth of perturbations.

This is not a peculiarity of one loss. The three canonical uses of a differentiable simulator all backpropagate through the rollout and so all carry the factor M_T: system identification differentiates the trajectory with respect to physical parameters, trajectory optimisation differentiates it with respect to a control sequence, and first-order (analytic) policy gradients differentiate it with respect to policy parameters [7, 6]. In each case the gradient inherits whatever M_T does.

What M_T does is set by λ_1. From the definition of the largest Lyapunov exponent, Eq. (5), the rollout Jacobian stretches a generic vector as $\|M_T\| \sim e^{\lambda_1 T}$, so on a chaotic system ($\lambda_1 > 0$) the

gradient magnitude grows exponentially in the horizon. The useful signal does not grow with it: the exponentially large contributions come from the most sensitive directions and inflate both the gradient and its variance, so beyond $T \sim 1/\lambda_1$ the analytic gradient is dominated by chaotic amplification rather than by the quantity one meant to optimise [8, 6, 7]. This is the *curse of chaos* for differentiable simulators, and it is the precise statement Part A measures: the same $1/\lambda_1$ horizon that bounds prediction also bounds the useful length of a differentiable rollout.

Equation (7) also gives a second, independent handle on λ_1. By Eq. (5), the growth rate of the gradient gain $\|M_T\|$ with T is itself a finite-horizon estimate of λ_1—distinct from the Benettin/QR estimator of Section 2.1, computed from the same orbit, and expected to agree with it as $T \to \infty$ by Oseledets. Section 3.3 makes this comparison the basis of Part A.

2.3 The three systems

We study three mechanical systems that share a common structure but span the range of dynamical behaviour we care about. Each is a conservative Hamiltonian system: writing the state as a configuration q with conjugate momentum p, the dynamics follow from a Hamiltonian

$$H(q, p) = \tfrac{1}{2}\, p^\top M(q)^{-1} p + V(q), \tag{8}$$

the sum of kinetic and potential energy, with $M(q)$ the configuration-dependent mass (inertia) matrix [9]. Equivalently, in second-order form the passive dynamics obey the manipulator equation $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = 0$ [10]; control enters as a forcing term on the right, but our Lyapunov diagnostics concern the underlying conservative flow, for which Liouville's theorem applies and the spectrum sums to zero as in Section 2.1.

The three systems form a deliberate ladder in λ_1:

- **Pendulum** — a single degree of freedom with separable Hamiltonian $H = \frac{1}{2}p^2 + V(\theta)$. A one-degree-of-freedom autonomous Hamiltonian is integrable: energy conservation confines every orbit to a level set of H, leaving no room for exponential divergence, so $\lambda_1 = 0$ [9]. This is our ordered baseline.
- **Cartpole** — a pole hinged on a translating cart: two degrees of freedom, underactuated. It is the canonical control benchmark [11, 10] and sits at intermediate complexity.
- **Acrobot** — a two-link pendulum actuated only at the elbow, the textbook chaotic underactuated system [12]. Its inertia matrix $M(q)$ depends on the configuration, making the Hamiltonian *non-separable*, and its passive dynamics are chaotic, $\lambda_1 > 0$.

The separable/non-separable distinction is the structural divide. When M is constant the kinetic term in Eq. (8) decouples from q, and the position and momentum updates can be taken independently; when $M(q)$ varies—as for the acrobot—the two are coupled. That coupling is both the origin of the acrobot's chaos and the reason it is the demanding case for the structure-preserving integrators and world models of Part B.

Because the consistency checks of Section 2.1 rely on volume preservation, the integrator must not manufacture or destroy phase-space volume of its own accord. We therefore discretise with semi-implicit (symplectic) Euler. For the two conservative systems whose Lyapunov spectrum we measure this preserves the symplectic structure and keeps the discrete one-step map volume-preserving with $\sum_i \lambda_i = 0$ [13, 14]—exactly for the separable pendulum ($\det J = 1$ to machine precision), and up to $O(dt^2)$ for the non-separable acrobot—whereas a generic explicit scheme (forward Euler, standard

Runge–Kutta) injects numerical dissipation or expansion that would bias the very spectrum we are trying to measure. The cartpole's larger-step discretisation is not volume-preserving, and it enters only as the forced swing-up demonstration of Part A, never in a Lyapunov claim. The step kernels are detailed in Section 3.1. The resulting ladder—$\lambda_1 = 0$ for the pendulum up to $\lambda_1 > 0$ for the acrobot—is the controlled axis along which Part A sweeps the gradient diagnostics and Part B contrasts a learned model's recovered exponent against the truth.

3 Methods — what we simulate and how

3.1 Differentiable Warp simulators

We implement each system of Section 2.3 directly as a Warp [5] compute kernel that advances the state by one *velocity-first semi-implicit (symplectic) Euler* step: the angular acceleration is evaluated at the current state, the velocity is updated first, and the configuration is then advanced with the *new* velocity,

$$\omega_{t+1} = \omega_t + dt\, a(q_t, \omega_t), \qquad q_{t+1} = q_t + dt\, \omega_{t+1}. \tag{9}$$

For a single degree of freedom this map is exactly symplectic; for the non-separable acrobot it is symplectic to $O(dt^2)$ (Section 2.3). The accelerations a are the only system-specific part:

- **Pendulum** (state (θ, ω)): $a = -(g/\ell)\sin\theta$.
- **Acrobot** (state $(\theta_1, \theta_2, \omega_1, \omega_2)$): the angular accelerations solve the manipulator equation $M(\theta)\,\dot{\omega} = F(\theta, \omega)$ with the point-mass double-pendulum inertia matrix and passive (Coriolis + gravity) forcing

  $$M(\theta) = \begin{bmatrix} (m_1 + m_2)\, l_1^2 & m_2 l_1 l_2 \cos(\theta_1 - \theta_2) \\ m_2 l_1 l_2 \cos(\theta_1 - \theta_2) & m_2 l_2^2 \end{bmatrix}, \quad F = \begin{bmatrix} -m_2 l_1 l_2\, \omega_2^2 \sin(\theta_1 - \theta_2) - (m_1 + m_2)\, g l_1 \sin\theta_1 \\ m_2 l_1 l_2\, \omega_1^2 \sin(\theta_1 - \theta_2) - m_2 g l_2 \sin\theta_2 \end{bmatrix}. \tag{10}$$

  The kernel advances Eq. (9) with $\dot{\omega} = M(\theta)^{-1} F$ in the closed form obtained by inverting the 2×2 system; we verified that this closed form reproduces $M(\theta)^{-1}F$ to machine precision. Because M depends on θ, the Hamiltonian is non-separable, which is the structural feature exploited in Part B (Section 5).
- **Cartpole** (state (x, θ, v, ω)): the uniform-rod benchmark [11, 10], with

  $$\ddot{\theta} = \frac{g \sin\theta - \cos\theta\, [(f + m_p \ell\, \omega^2 \sin\theta)/(m_c + m_p)]}{\ell(\frac{4}{3} - m_p \cos^2\theta/(m_c + m_p))}, \qquad \ddot{x} = \frac{f + m_p \ell\, \omega^2 \sin\theta}{m_c + m_p} - \frac{m_p \ell\, \ddot{\theta} \cos\theta}{m_c + m_p}, \tag{11}$$

  driven by the cart force f. The cartpole is used only as the trajectory-optimisation target of Part A (the forced swing-up demonstration); over Lyapunov horizons its discretisation is *not* volume-preserving (the measured mechanical energy drifts by $\approx 1.8\%$ over 20 steps at $dt = 0.02$), so it is excluded from all Lyapunov claims.

Reverse-mode differentiation requires the full trajectory to be available on the backward pass, so each rollout is stored as a $(T{+}1) \times \dim$ array, writing one new row per step and never overwriting a cell; the adjoint then walks back along the stored states, multiplying in one one-step Jacobian J_t per step exactly as in Eq. (7). All simulators run on CPU, single-threaded. The integration step and physical parameters are collected in Table 1.

System	dt	Parameters
Pendulum	5×10^{-3}	$g = 9.81,\ \ell = 1$
Cartpole	2×10^{-2}	$m_c = 1,\ m_p = 0.1,\ \ell = 0.5,\ g = 9.81$
Acrobot	5×10^{-4}	$m_1 = m_2 = 1,\ l_1 = l_2 = 1,\ g = 9.81$

Table 1: Integration step and physical parameters of the three Warp simulators.

3.2 The Lyapunov estimator and its calibration

We operationalize the Benettin/QR estimator of Section 2.1 as follows. From a seeded random orthonormal frame $Q \in \mathbb{R}^{n\times k}$ (the k leading exponents; $k = n$ for the full spectrum), we advance the tangent frame one step at a time, re-orthonormalising through a thin QR factorisation $J_t Q = Q'R$ at *every* step, accumulating $\ln|R_{ii}|$, and dividing the running sums by the elapsed time $N\,dt$ to obtain the exponents in physical (per-unit-time) units, sorted in descending order (the telescoping that makes $\frac{1}{N\,dt}\sum_t \ln R_{ii}^{(t)} \to \lambda_i$ exact is spelled out in Appendix A). The initial transient is discarded by slicing the trajectory before the average: for the dissipative Lorenz calibration this is the relaxation onto the strange attractor (the first 2000 steps), and for every system it is the alignment of the QR frame onto the leading Oseledets directions (Section 3.3). A single such estimator is used for every result in this paper.

We calibrate it against two systems with known answers.

- **Harmonic oscillator** (an exact rotation map, all exponents zero): the estimator returns $|\lambda_i| \lesssim 2 \times 10^{-15}$ over 2×10^4 steps—zero to machine precision, as expected since each step contributes $\ln|\pm 1| = 0$ exactly.
- **Lorenz** ($\sigma = 10,\ \rho = 28,\ \beta = 8/3$; RK4 at $dt = 5 \times 10^{-3}$, first 2000 steps dropped): the finite-time largest exponent is itself a fluctuating quantity. Single-trajectory estimates scatter by ~ 0.02 around the standard literature value $\lambda_1 = 0.9056$ over a few hundred Lyapunov times, and the running average on a long trajectory converges to it ($\lambda_1 = 0.905$ at 1.6×10^3 Lyapunov times). This scatter is intrinsic finite-time variance, not a Jacobian or estimator bias: finite-difference and analytic (variational-RK4) Jacobians agree on λ_1 to 3×10^{-8} along a common orbit. As a sharp structural check valid at *every* horizon, the recovered spectrum has a near-zero middle exponent and sums to $\sum_i \lambda_i = -13.667$, matching the constant flow divergence $\operatorname{tr} Df = -(\sigma + 1 + \beta)$ to machine precision (Eq. (6)).

Finally, the per-step Jacobian feeding the estimator is computed analytically for the pendulum and by central finite differences for the acrobot, and each is cross-checked against a third, independent route—Warp's reverse-mode autodiff. The analytic pendulum Jacobian agrees with autodiff to 1.7×10^{-9} and the central-difference acrobot Jacobian to 2.0×10^{-5}, confirming that the tangent dynamics the estimator integrates are the ones the simulator actually propagates.

3.3 Two estimators of λ_1

Part A rests on comparing two finite-horizon estimates of λ_1 computed from the *same* orbit. The first is the calibrated Benettin/QR estimator of Section 3.2. The second reads λ_1 off the growth of the gradient gain introduced in Section 2.2: since $\|M_T\| \sim e^{\lambda_1 T}$, the least-squares slope of $\ln \|M_T\|_2$ against T is itself an estimate of λ_1. Here $\|M_T\|_2$ is the spectral norm (top singular value) of the rollout Jacobian, and the slope is fit over a range of horizons reaching several Lyapunov times (T

from $\sim 10^3$ up to 1.6×10^4 integration steps).

These two numbers are distinct finite-T functionals of the same Jacobian sequence, *not* algebraically identical: the QR estimator averages the local logarithmic stretching of an orthonormal frame, whereas the slope tracks the global spectral norm of the accumulated product. Both converge to the true λ_1 as $T \to \infty$ by Oseledets' theorem, but only once $T \gg 1/(\lambda_1 - \lambda_2)$—the time a generic direction needs to align with the leading one —and the slope is the higher-variance of the two. Part A therefore reports their agreement as a *consistency relation*—evidence that the gradient-gain growth rate and the dynamical λ_1 are one and the same number—rather than as an exact identity.

4 Part A — λ_1 bounds differentiable-simulator gradients

Part A establishes the first pillar of the thesis: the largest Lyapunov exponent λ_1 governs the gradients a differentiable simulator produces. Section 2.2 showed that every such gradient carries the rollout-Jacobian factor M_T with $\|M_T\| \sim e^{\lambda_1 T}$, and Section 3.3 set up a second, gradient-side estimate of λ_1 read off that growth rate.

Three systems play distinct roles. The chaotic *acrobot* is the workhorse and carries every λ_1 claim; the integrable *pendulum* ($\lambda_1 \approx 0$) is the null contrast, showing the effect vanishes when the dynamics are not chaotic; and the forced *cartpole*, whose λ_1 is ill-defined (Section 2.3), appears only as a downstream application with no Lyapunov claim of its own.

Against this cast we (i) confirm the growth law and that its rate *is* λ_1, on the matched pendulum–acrobot pair; (ii) show it holds across the whole integrable–chaotic range, by sweeping the acrobot's energy so λ_1 climbs continuously from near zero; (iii) turn the growth law into a *usefulness* limit—the gradient predictability horizon $T \lesssim 1/\lambda_1$—through the collapse of the analytic-gradient signal-to-noise ratio on the acrobot; and (iv) draw the consequence for trajectory optimisation, swinging up the cartpole by gradient descent *inside* that horizon.

4.1 The gradient-gain growth law

This subsection operationalises the bridge of Section 2.2—that a differentiable simulator's gradient carries the factor $\|M_T\| \sim e^{\lambda_1 T}$—and reads its rate off the slope estimator of Section 3.3. For each system we plot $\ln \|M_T\|_2$ against the horizon T (Fig. 1); the slope is the gradient-gain rate. On the chaotic acrobot the gain grows exponentially at $1.07\,\mathrm{s}^{-1}$, matching the independently measured Benettin $\lambda_1 = 1.18\,\mathrm{s}^{-1}$; on the integrable pendulum both rates vanish ($\lambda_1 \approx 0.08$, slope $\approx 0.06\,\mathrm{s}^{-1}$) and the gain is sub-exponential. The gradient gain thus blows up *at the Lyapunov rate* on chaotic dynamics, and not at all on integrable ones.

Two points fix the interpretation. First, the rate is a *single* factor of λ_1: we fit the Jacobian norm $\|M_T\|_2$, which grows as $e^{\lambda_1 T}$, so its log-slope is λ_1; a squared quantity—a squared-loss objective, or the squared separation $\|\delta x_T\|^2$—would grow as $e^{2\lambda_1 T}$ and read $2\lambda_1$ instead. Second, the coincidence of the two rates is the *consistency relation* of Section 3.3, not an identity: slope and Benettin exponent are exact equals only for a constant-Jacobian map, and on the acrobot they agree to within the finite-time scatter of the higher-variance slope ($\approx 0.1\,\mathrm{s}^{-1}$, about 10%).

4.2 The law holds across regimes

Section 4.1 fixed the law on two orbits; here we vary the dynamics continuously. We sweep the acrobot across initial energies (θ_1 from 0.5 to 3.0, raising λ_1 from near zero to $\sim 1.2\,\mathrm{s}^{-1}$) and add

ain $\|\partial x_T/\partial x_0\|$ blows up exponentially at λ_1 for chaotic dynamics (acrobot), but not for integrable dynamics (

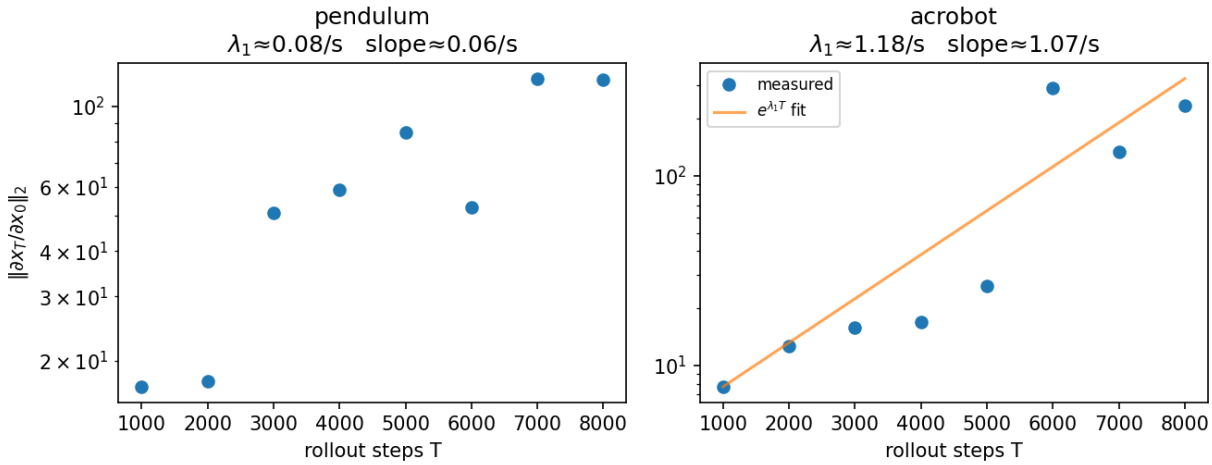

Figure 1: Gradient gain $\|\partial x_T/\partial x_0\|_2$ versus rollout horizon T (semilog). Integrable pendulum (left): sub-exponential, $\lambda_1 \approx 0$. Chaotic acrobot (right): exponential growth whose log-slope ($1.07\,\mathrm{s}^{-1}$) matches the Benettin λ_1 ($1.18\,\mathrm{s}^{-1}$).

the integrable pendulum, and for each we compare the gradient-gain slope against the Benettin λ_1 (Fig. 2). The points cluster on the line slope $= \lambda_1$: across the integrable-to-chaotic range the two rates correlate at 0.99, with best-fit slope 0.96 (ideal 1) and near-zero intercept. The gradient-gain rate is thus λ_1 not merely on a single orbit but as a function of the regime.

The cartpole is deliberately absent: it is forced and not volume-preserving (Section 2.3), so its λ_1 is regime-dependent and ill-defined, and the acrobot energy sweep already spans the integrable-to-chaotic transition. The horizontal error bars report the finite-time scatter of the Benettin estimate over independent windows—an honest depiction of finite-time Lyapunov uncertainty, not a confidence interval on the plotted point.

4.3 The gradient predictability horizon

The growth law has a sharp operational consequence. Section 2.2 argued that because $\|M_T\|$ inflates both a gradient and its variance, beyond $T \sim 1/\lambda_1$ the analytic gradient of a differentiable rollout is dominated by chaotic amplification rather than by the objective—the *curse of chaos*. We make this quantitative with a signal-to-noise ratio: perturbing the initial state over an ensemble of nearby conditions, we compare the mean analytic gradient $\nabla_{x_0} \|x_T\|^2$ to its spread across the ensemble (Fig. 3). On the acrobot ($\lambda_1 = 1.23\,\mathrm{s}^{-1}$) the SNR is large well inside the horizon ($\approx$170 at $T\lambda_1 \approx 0.3$) and falls by roughly an order of magnitude past $T\lambda_1 \approx 1$ (to $\approx$20–50): the gradient still has a large magnitude, but its *direction* is no longer shared across nearby initial conditions. The crossover sits at the predictability horizon $T \sim 1/\lambda_1$, up to a logarithmic factor $\sim \log(1/\varepsilon)/(\lambda_1\, dt)$ set by the perturbation scale ε.

This collapse is genuinely nonlinear, not a normalisation artifact. Each ensemble gradient has magnitude $\sim \|M_T\| \sim e^{\lambda_1 T}$, so the ensemble variance grows as $e^{2\lambda_1 T}$; on a purely linear map this common growth cancels between the mean (signal) and the spread (noise) and the SNR stays flat. The observed decline therefore measures real loss of *directional* agreement among neighbouring

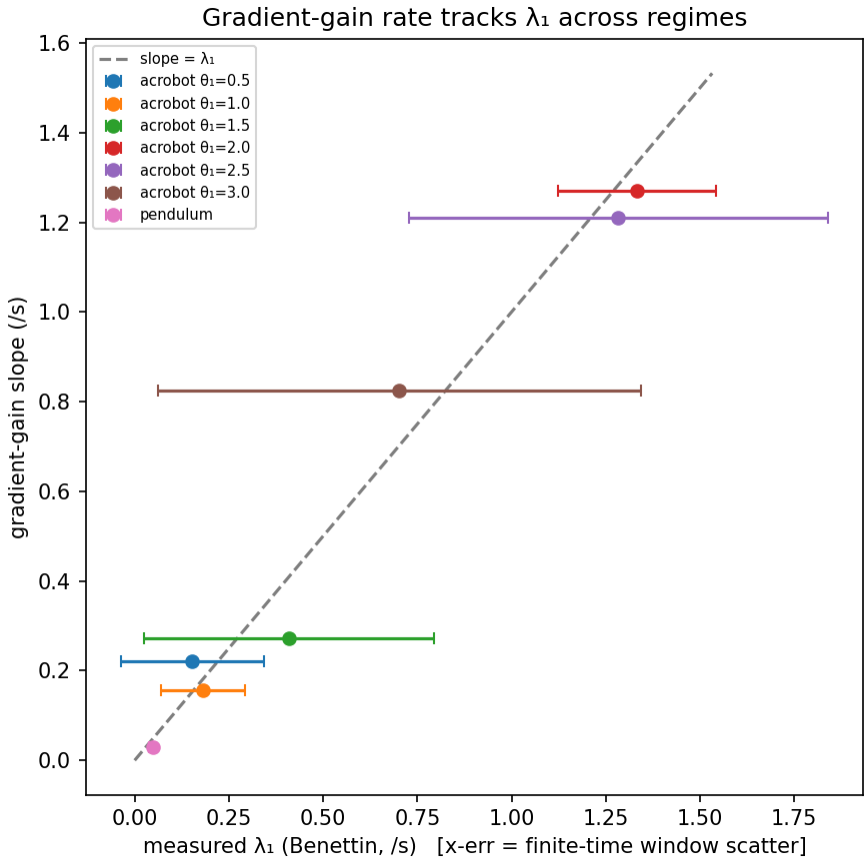

Figure 2: Gradient-gain slope versus Benettin λ_1 across an acrobot energy sweep ($\theta_1 = 0.5 \to 3.0$) and the pendulum; points cluster on $y = x$ (correlation 0.99). Horizontal bars are finite-time Lyapunov window scatter, not confidence intervals.

trajectories: past $T \sim 1/\lambda_1$ the analytic gradient points a different way for every nearby start, which is exactly what makes long differentiable rollouts unusable for first-order optimisation.

4.4 Consequence: optimisation through the simulator

Sections 4.1–4.3 measured how far a differentiable rollout's gradient stays usable; this last subsection shows the positive side of that boundary—that *inside* the horizon the gradient is good enough to optimise through. We swing a cartpole from hanging to upright by gradient descent on the control sequence, backpropagating the upright-tracking cost through the differentiable Warp rollout (Adam, horizon $T = 100$ steps; Fig. 4). The cost falls from 200.7 to 57.9 over 300 iterations—the analytic gradient, taken over a horizon short enough to remain informative, solves the task.

The cartpole appears only here, as an application: it is forced and not volume-preserving (Section 2.3), so it carries no Lyapunov claim. The point is not a number for this system but the converse of the curse of chaos—that the same gradient which becomes unusable past $T \sim 1/\lambda_1$ is, within that horizon, exactly what makes differentiable simulation a powerful optimiser.

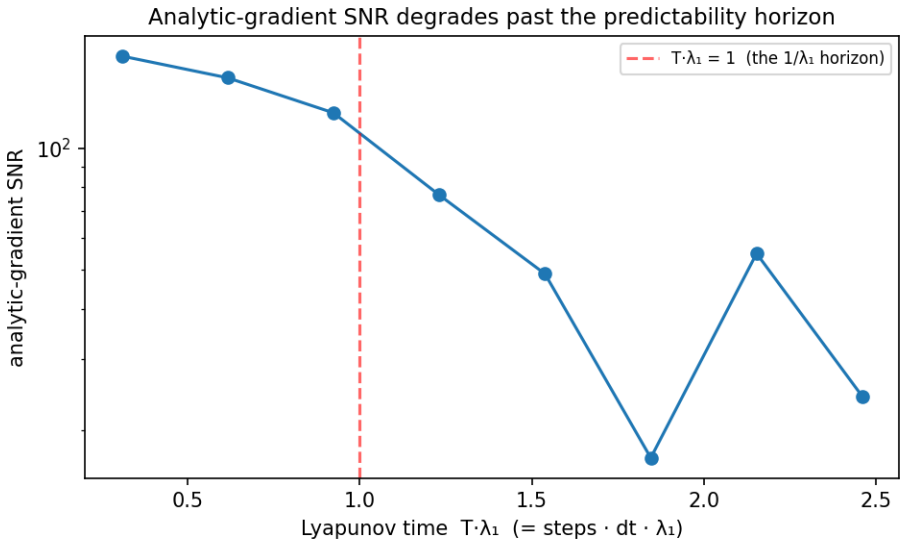

Figure 3: Analytic-gradient SNR versus Lyapunov time $T\lambda_1$ (acrobot). The SNR degrades past the predictability horizon $T\lambda_1 = 1$ (dashed): the gradient's direction decorrelates across nearby initial conditions even though its magnitude keeps growing.

5 Part B — Lyapunov fidelity of learned world models

Part A established that the largest Lyapunov exponent λ_1 governs the gradients a differentiable simulator produces when the physics is correct. Part B asks the dual question for a model that is *learned* rather than given: does a world model trained only to predict the next state inherit the right λ_1? This is the spine of the section. A multilayer perceptron fit by next-step regression reproduces the integrable exponent but *over-amplifies* the chaotic one—because next-step accuracy never constrains the Jacobian that carries λ_1—and only a model built with the right *hard* structural constraint, symplectic and volume-preserving, recovers it.

We reuse the cast of Part A: the integrable *pendulum* as the null contrast and the chaotic *acrobot* as the workhorse; the forced cartpole carries no Lyapunov claim and plays no part here. The argument runs in four steps: the learned model and how fast its rollout drifts from the truth (Section 5.1); the learned-versus-true λ_1 that its Jacobian implies (Section 5.2); why next-step training is blind to λ_1 in the first place (Section 5.3); and which structural constraint repairs it (Section 5.4).

5.1 The learned world model and its prediction drift

A world model replaces the simulator's one-step map $x_{t+1} = F(x_t)$ with a learned surrogate g_ϕ, fit so that iterating it reproduces trajectories of the true system. We deliberately use the smallest model that exhibits the effect: a residual multilayer perceptron $g_\phi(x) = x + \mathrm{net}_\phi(x)$ with two tanh hidden layers, trained by next-step mean-squared error on rollouts of the true simulator. It is given only the state pairs (x_t, x_{t+1})—no equations of motion, no Hamiltonian, no conserved quantity—which is exactly the regime in which one hopes a generic learner will simply absorb the dynamics from data.

Such a model can be accurate one step at a time yet useless over a rollout, so the first question is how quickly its predictions drift. We roll g_ϕ forward from a shared initial condition and track the prediction error $\|\hat{x}_t - x_t\|$ against the true trajectory (Fig. 5). On both systems the error grows, but

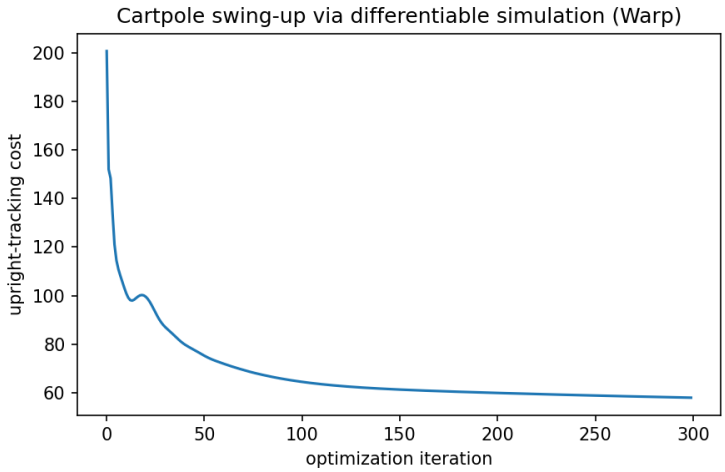

Figure 4: Cartpole swing-up by gradient descent through the differentiable simulator: upright-tracking cost versus optimisation iteration ($T = 100$, Adam). The cartpole is forced and is excluded from all λ_1 claims (Section 2.3).

at visibly different rates: on the integrable pendulum it rises slowly, while on the chaotic acrobot it climbs roughly twice as fast before saturating. A log-linear fit over the pre-saturation window gives effective rates of $0.96\,\mathrm{s}^{-1}$ (pendulum) and $1.74\,\mathrm{s}^{-1}$ (acrobot).

These rates measure *model fidelity*, not the Lyapunov exponent. Two effects are superimposed: the model's own approximation error, which accumulates even on the non-chaotic pendulum, and—on the acrobot—the genuine chaotic amplification of that error. The pendulum makes the distinction concrete: its fitted $0.96\,\mathrm{s}^{-1}$ is nowhere near its true $\lambda_1 \approx 0$, so the drift rate cannot be read as λ_1. Figure 5 is therefore a qualitative statement—prediction degrades faster on chaotic dynamics—and isolating the exponent the model has actually learned requires probing its Jacobian directly, the subject of Section 5.2.

5.2 Learned versus true λ_1

To read off the exponent the model has learned, we apply the *same* Benettin/QR estimator used for the true systems (Section 3.2) to the model's one-step Jacobian $\partial g_\phi / \partial x$, obtained by automatic differentiation. The only subtlety is where to evaluate it: rolling the model out and estimating λ_1 along its *own* trajectory would conflate an inaccurate Jacobian with a drifted orbit, so we instead evaluate the learned Jacobian along the *true* trajectory, after the same transient. Estimator, orbit, and transient are then identical for the learned and reference exponents; only the source of the Jacobian differs—true physics versus learned model—so any gap is the model's Jacobian alone. A faithful model, whose Jacobians match the true ones along the orbit, would return the true λ_1.

Figure 6 plots learned against true λ_1 for the two systems. On the integrable pendulum the model lands on the truth: a learned $\lambda_1 \approx 0.046\,\mathrm{s}^{-1}$ against a true $0.063\,\mathrm{s}^{-1}$, both effectively zero—the network correctly reports non-chaotic dynamics. On the chaotic acrobot it does not: the learned exponent is $3.45\,\mathrm{s}^{-1}$ against a true $1.04\,\mathrm{s}^{-1}$, an over-amplification by a factor of 3.3. The model is not merely imprecise here; it manufactures sensitivity the true dynamics do not have.

The asymmetry is the point. The network is trained to low next-step error on both systems, yet it

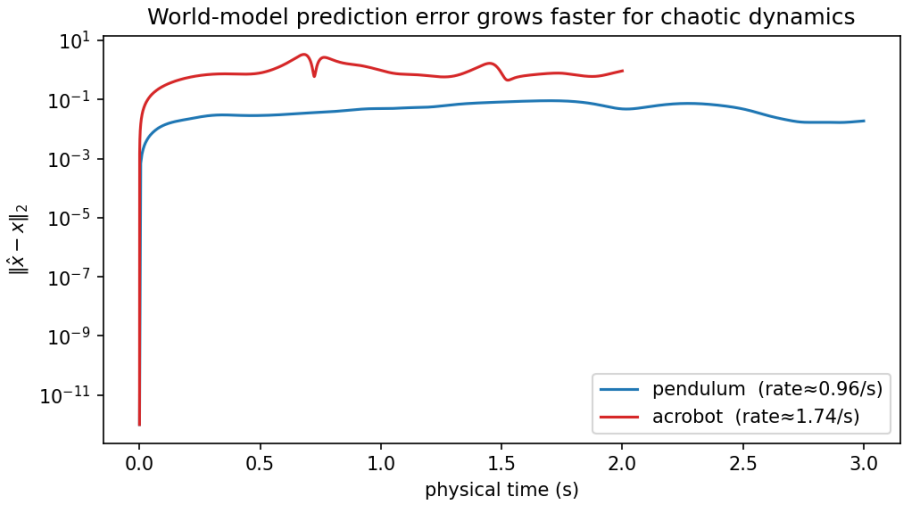

Figure 5: Residual-MLP world-model prediction error $\|\hat{x}_t - x_t\|$ versus physical time (semilog). Error grows on both systems but faster on the chaotic acrobot (effective log-slope $1.74\,\mathrm{s}^{-1}$) than on the integrable pendulum ($0.96\,\mathrm{s}^{-1}$). These are model-fidelity rates, not λ_1: the pendulum's is far from its true $\lambda_1 \approx 0$. The Lyapunov comparison is Section 5.2.

reproduces λ_1 only where the exponent is trivial and inflates it precisely where chaotic sensitivity makes it matter. Low next-step error is therefore no guarantee of dynamical fidelity: λ_1 is a property the training objective never sees, and on chaotic dynamics the learned model overstates it threefold. Section 5.3 explains why next-step regression is blind to it.

5.3 Why next-step training cannot see λ_1

The failure in Section 5.2 has a precise cause. Next-step mean-squared error penalises $\|g_\phi(x_t) - x_{t+1}\|^2$ at the training states: it supervises *where* points are mapped. But λ_1 is a property of the *Jacobian* $\partial g_\phi / \partial x$, which governs how infinitesimal perturbations are stretched, and next-step error never targets a derivative. With finite data and finite capacity a model can drive its pointwise error low while its Jacobian—constrained by no derivative target—remains substantially wrong. The threefold over-amplification of Section 5.2 is exactly that gap made visible: the network reproduces the trajectory well yet equips it with the wrong sensitivity.

There is a sharper way to name what is missing. The true dynamics are Hamiltonian, and in canonical coordinates they conserve phase-space volume exactly—the Lyapunov spectrum sums to zero (Eq. (6)). That volume law is a structural fingerprint of genuine mechanics, yet it is a property of the Jacobian's *determinant*—once more, the object next-step regression never touches—so a pointwise learner has no route to discover it. We read this fidelity off the coordinate-invariant exponent λ_1 rather than a raw volume sum, since the volume law is exact only in the canonical (θ, p) frame and not the (θ, ω) coordinates the model is fit in. The same reasoning points to the remedy: rather than hope training stumbles onto the right Jacobian, build the structure in. Section 5.4 does exactly this, contrasting a soft penalty that *asks* for volume preservation with a symplectic architecture that *guarantees* it.

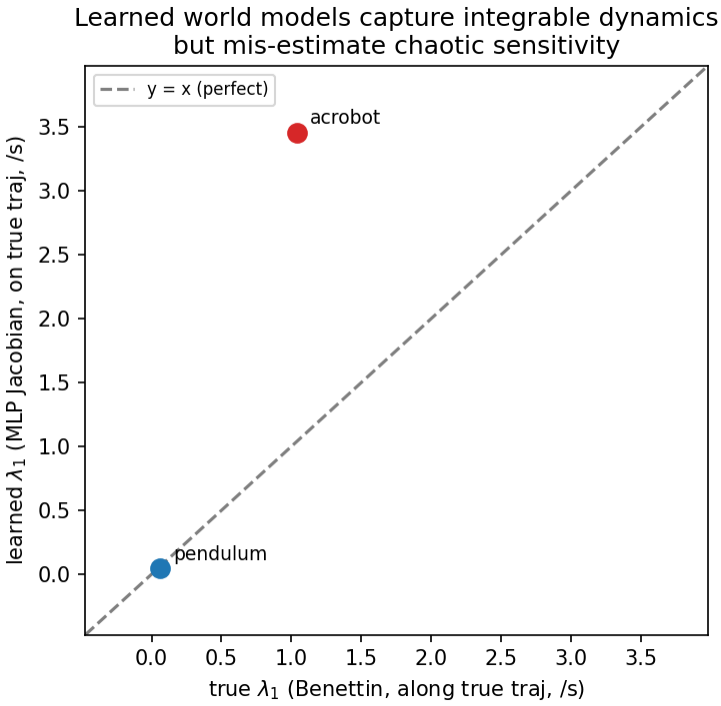

Figure 6: Learned versus true λ_1, both from the same Benettin/QR estimator evaluated along the true orbit. The integrable pendulum sits on the $y = x$ line (both $\lambda_1 \approx 0$); the chaotic acrobot lies far above it—learned $3.45\,\mathrm{s}^{-1}$ versus true $1.04\,\mathrm{s}^{-1}$, an over-amplification of $3.3\times$. Next-step accuracy does not imply the right λ_1.

5.4 Structure-preserving world models

Section 5.3 argued that the cure is to build the missing structure into the model rather than hope training discovers it. We test this on the chaotic acrobot with three world models that impose volume structure with increasing force. The first is the plain MLP of Sections 5.1–5.3, with no constraint. The second is a *volume-penalty* MLP, which adds to the next-step loss a soft penalty $\left(\log|\det \partial g_\phi/\partial x|\right)^2$ pulling the one-step Jacobian toward unit determinant—an explicit *request* for volume preservation. The third is a *Hamiltonian neural network* (HNN), which builds it in structurally, through a symplectic integration step.

The HNN is the structurally honest model. It learns a scalar Hamiltonian $H_\phi(q, p)$ on canonical coordinates ($q = \theta$, $p = M(\theta)\,\omega$, with M the mass matrix), forms the Hamiltonian flow $\dot{q} = \partial H/\partial p$, $\dot{p} = -\partial H/\partial q$, and advances it with a symplectic Euler step; angles enter through a $(\cos\theta, \sin\theta)$ embedding to respect the circle topology. For the acrobot's non-separable Hamiltonian this step is volume-preserving to $O(dt^2)$—its Jacobian has unit determinant up to that order—so the model conserves phase-space volume essentially by construction, not by training; measured along the canonical image of the true orbit, its spectrum sums to $0.001\,\mathrm{s}^{-1}$, against the true zero. Because λ_1 is coordinate-invariant, that same canonical evaluation gives a value directly comparable to the true (θ, ω) exponent.

Figure 7 reports the learned λ_1 of each model against the true acrobot exponent ($\approx 1.14\,\mathrm{s}^{-1}$ on this orbit). The two MLPs over-amplify, as in Section 5.2: the plain MLP gives $2.47\,\mathrm{s}^{-1}$ and

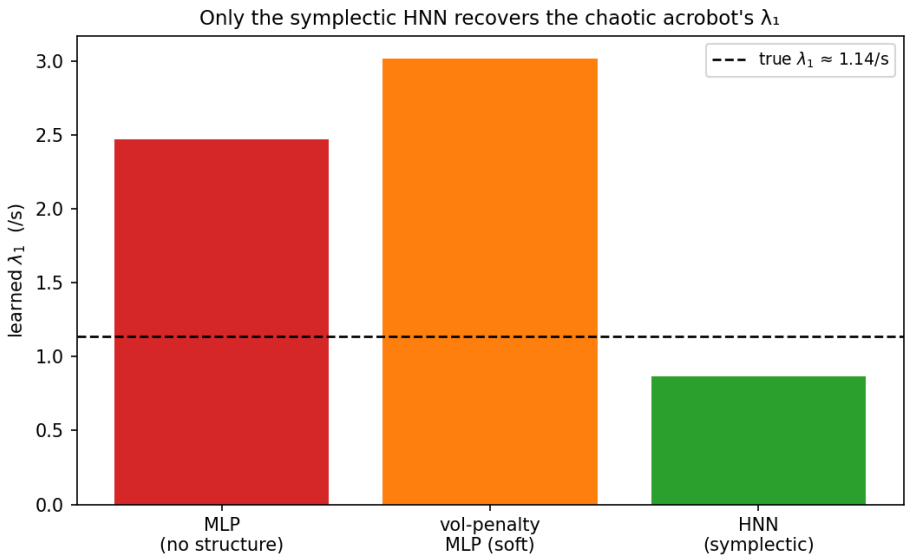

Figure 7: Learned λ_1 on the chaotic acrobot for three world models, against the true exponent (dashed, $\approx 1.14\,\mathrm{s}^{-1}$). The unconstrained MLP ($2.47\,\mathrm{s}^{-1}$) and the soft volume-penalty MLP ($3.02\,\mathrm{s}^{-1}$) over-amplify; only the symplectic HNN ($0.87\,\mathrm{s}^{-1}$), volume-preserving by construction, lands near the truth—undershooting by about 24%. Only a hard structural constraint recovers λ_1.

the volume-penalty MLP $3.02\,\mathrm{s}^{-1}$—the soft penalty does not close the gap, and here is no better than no penalty at all. Only the HNN lands near the truth, at $0.87\,\mathrm{s}^{-1}$: the sole model that does not manufacture spurious sensitivity. The lesson is that the constraint must be hard—*asking* for volume preservation through a penalty is not enough, while *guaranteeing* it through a symplectic architecture is.

The win is qualified. The HNN recovers the right magnitude and the volume structure, but it undershoots the true exponent by about 24% (0.87 against $1.14\,\mathrm{s}^{-1}$). Structure preservation is therefore necessary but not, on its own, sufficient for an exact match—a gap we leave to the Limitations (Section 7), along with the finite-time, single-seed, and integrator dependence of these estimates, the true reference itself included.

Part B thus mirrors Part A under one exponent. Where Part A showed that λ_1 bounds the gradients a correct simulator yields, Part B shows that a learned world model is only as faithful as the λ_1 it reproduces—and that next-step accuracy does not deliver it. A model can predict the next state almost perfectly and still misjudge the chaotic sensitivity of the system it represents; λ_1 is the diagnostic that exposes the difference, and a structural prior is what repairs it.

6 Extensions

The diagnostics here are deliberately demonstrated on low-dimensional, single-threaded CPU systems. The natural next steps push them onto GPU hardware and onto the large learned models of the surrounding Physical-AI stack.

- **Scale to GPU.** Every diagnostic in this paper—the Benettin/QR estimator, the gradient-gain slope, and the perturbed-rollout SNR—is a sweep or ensemble average over many *independent*

trajectories (initial-condition averaging, the energy sweep, the SNR ensemble). The outer loop over trajectories, seeds, and sweep points is therefore *embarrassingly parallel* and maps directly onto the GPU—in the spirit of the thousands of parallel environments used for large-scale robot learning—with only the per-trajectory time-stepping remaining sequential. Re-implementing the Warp simulators on **Newton** (GPU; the production successor to the removed `warp.sim`) is the concrete path, and the prerequisite for everything below.

- **Cosmos / large world models.** The Part-B fidelity test extends from our small surrogate to a large video world model such as **Cosmos**: does it preserve the physical Lyapunov spectrum, or hallucinate (and wash out) sensitivity? The error-growth diagnostic (Section 5.1) transfers to pixel space as a qualitative probe; the full Lyapunov-fidelity diagnostic (Section 5.2) requires recovering physical state (θ, ω) from generated video—a perception front-end—or a latent-space tangent-vector adaptation, both research problems rather than a front-end swap, and both demanding GPU-scale compute.
- **GR00T / action models.** For a closed-loop policy such as **GR00T**, the natural robustness diagnostic is the finite-time Lyapunov exponent of the observation $\to$ action $\to$ next-state map, with the policy in a simulation loop (Isaac Lab / Newton) and numerical (perturbed-rollout) sensitivities, since a large policy with a finite context window is a non-Markovian, non-autodiff-friendly map.
- **Contact and stiff dynamics.** A distinct gradient-pathology regime—non-smooth rather than chaotic—deliberately outside this conservative CPU study, but the setting where differentiable-simulator gradients most often break in practice.

7 Limitations

[To be written.]

A The Benettin/QR estimator: telescoping and the log-sum

Sections 2.1 and 3.2 assert that averaging the logarithms of the QR diagonals recovers the spectrum; we record here why, since the running sum is never formed explicitly in the main text.

Let $J_0, \dots, J_{N-1}$ be the one-step Jacobians along an orbit and $Q^{(0)} \in \mathbb{R}^{n\times k}$ the initial orthonormal frame ($Q^{(0)\top} Q^{(0)} = I_k$). Each step performs a thin QR factorisation and feeds the resulting frame to the next step,

$$J_t\, Q^{(t)} = Q^{(t+1)} R^{(t+1)}, \qquad t = 0, \dots, N-1, \tag{12}$$

with $Q^{(t+1)}$ orthonormal-columned and $R^{(t+1)}$ upper triangular with positive diagonal. Substituting (12) repeatedly telescopes the rollout Jacobian $M_N = \prod_{t<N} J_t$ acting on the initial frame,

$$M_N\, Q^{(0)} = J_{N-1} \cdots J_0\, Q^{(0)} = Q^{(N)}\, \underbrace{R^{(N)} R^{(N-1)} \cdots R^{(1)}}_{\mathcal{R}_N}. \tag{13}$$

Equation (13) is itself a thin QR factorisation of $M_N Q^{(0)}$: the intermediate frames are not cancelled but carried forward as each step's input, and the entire accumulated rotation collects in the single factor $Q^{(N)}$. Two properties then isolate the spectrum.

First, $\mathcal{R}_N$ is a product of upper-triangular matrices, hence upper triangular with diagonal equal to the product of diagonals,

$$(\mathcal{R}_N)_{ii} = \prod_{t=1}^{N} R_{ii}^{(t)} \qquad \Longrightarrow \qquad \ln(\mathcal{R}_N)_{ii} = \sum_{t=1}^{N} \ln R_{ii}^{(t)}, \tag{14}$$

converting the overflow-prone matrix product into the additive running sum the estimator accumulates—the entire numerical reason for re-orthonormalising at every step rather than forming M_N once.

Second, $Q^{(N)}$ has orthonormal columns and is therefore an isometry: $\left\|Q^{(N)}x\right\| = \|x\|$, its singular values are all one, $\left\|Q^{(N)}\right\| = 1$, and so $\sigma_i(M_N Q^{(0)}) = \sigma_i(\mathcal{R}_N)$. It rotates the frame but neither stretches nor shrinks it, contributing nothing to any growth rate. Dividing (14) by the elapsed physical time $N\,dt$ and letting $N \to \infty$ gives

$$\lambda_i = \lim_{N\to\infty} \frac{1}{N\,dt} \sum_{t=1}^{N} \ln R_{ii}^{(t)}, \tag{15}$$

with convergence guaranteed by Oseledets' theorem for a generic $Q^{(0)}$. "Generic" is what the seeded *random* frame of Section 3.2 secures: with probability one the columns of $Q^{(0)}$ avoid the measure-zero subspaces of the Oseledets filtration that grow more slowly than the top-k rates, so (15) returns $\lambda_1 \geq \cdots \geq \lambda_k$ rather than a slower set. That the *diagonal* of $\mathcal{R}_N$ suffices—rather than its singular values—is exactly because the isometry $Q^{(N)}$ ties the two together.

References

[1] Valery I. Oseledets. "A Multiplicative Ergodic Theorem. Lyapunov Characteristic Numbers for Dynamical Systems". In: *Transactions of the Moscow Mathematical Society* 19 (1968), pp. 197–231.

[2] Jean-Pierre Eckmann and David Ruelle. "Ergodic Theory of Chaos and Strange Attractors". In: *Reviews of Modern Physics* 57.3 (1985), pp. 617–656.

[3] Giancarlo Benettin et al. "Lyapunov Characteristic Exponents for Smooth Dynamical Systems and for Hamiltonian Systems; a Method for Computing All of Them". In: *Meccanica* 15 (1980), pp. 9–30.

[4] Rainer Engelken, Fred Wolf, and L. F. Abbott. "Lyapunov Spectra of Chaotic Recurrent Neural Networks". In: *Physical Review Research* 5 (2023), p. 043044.

[5] Miles Macklin. *Warp: A High-performance Python Framework for GPU Simulation and Graphics*. NVIDIA GPU Technology Conference (GTC). 2022. URL: `https://github.com/NVIDIA/warp`.

[6] Paavo Parmas et al. "PIPPS: Flexible Model-Based Policy Search Robust to the Curse of Chaos". In: *Proceedings of the 35th International Conference on Machine Learning (ICML)*. PMLR. 2018.

[7] Hyung Ju Terry Suh et al. "Do Differentiable Simulators Give Better Policy Gradients?" In: *Proceedings of the 39th International Conference on Machine Learning (ICML)*. PMLR. 2022.

[8] Luke Metz et al. "Gradients Are Not All You Need". In: *arXiv preprint arXiv:2111.05803* (2021).

[9] Vladimir I. Arnold. *Mathematical Methods of Classical Mechanics*. 2nd ed. Vol. 60. Graduate Texts in Mathematics. Springer, 1989.

[10] Russ Tedrake. *Underactuated Robotics: Algorithms for Walking, Running, Swimming, Flying, and Manipulation*. Course Notes for MIT 6.832. 2023. URL: `https://underactuated.mit.edu`.

[11] Andrew G. Barto, Richard S. Sutton, and Charles W. Anderson. "Neuronlike Adaptive Elements That Can Solve Difficult Learning Control Problems". In: *IEEE Transactions on Systems, Man, and Cybernetics* SMC-13.5 (1983), pp. 834–846.

[12] Mark W. Spong. "The Swing Up Control Problem for the Acrobot". In: *IEEE Control Systems Magazine* 15.1 (1995), pp. 49–55.

[13] Ernst Hairer, Christian Lubich, and Gerhard Wanner. *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*. 2nd ed. Vol. 31. Springer Series in Computational Mathematics. Springer, 2006.

[14] Benedict Leimkuhler and Sebastian Reich. *Simulating Hamiltonian Dynamics*. Cambridge Monographs on Applied and Computational Mathematics. Cambridge University Press, 2004.

[15] Sam Greydanus, Misko Dzamba, and Jason Yosinski. "Hamiltonian Neural Networks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 32. 2019.